Level 5 · about 35 h · Python 3.8+

Project: A Note class plus a one-line import check
Examples run on CPython 3.12.3. **You will be able to:**

- Define a class with an __init__ method and read an attribute
- Build one instance and call a method on it
- Explain that import runs a module once and stores it on sys.modules
- Say what a virtual environment isolates

Classes and Modules

A class bundles data and methods

The tutorial describes a class as a way to bundle data and the operations on that data. `__init__` runs when you call the class. `self` is the instance.

```
class Note:
    def __init__(self, text):
        self.text = text

    def shout(self):
        return self.text.upper()

note = Note("hello")
print(note.text)
print(note.shout())
```

```
Output (CPython 3.12.3):
hello
HELLO
```

import runs the module once

`import` executes a module and keeps the module object in `sys.modules`. Importing again returns that same object. `json` is in the standard library, so this sample does not install anything.

```
import json
import sys
print("json" in sys.modules)
print(json.dumps({"ok": True}))
```

```
Output (CPython 3.12.3):
True
{"ok": true}
```

A virtual environment isolates installs

venv creates an environment with its own site-packages. PEP 405 is the design. This page does not create an environment during the build, because that would write a folder. The check below only reports that the venv module can be found.

```
import importlib.util
print(importlib.util.find_spec("venv") is not None)
```

```
Output (CPython 3.12.3):
True
```

Check yourself

1. When does `__init__` run?
2. Where does Python keep a module after the first import?
3. What does a virtual environment keep separate?

Sources

- Python tutorial: Classes
- Python tutorial: Modules
- Python docs: venv
- PEP 405 – Python Virtual Environments

Code © CypherMonkeyLabs, MIT. Text CC BY-NC-SA 4.0. Educational material, not accredited coursework; no outcome or income promised. Built 2026-10-18.